

One important gotcha to keep in mind here is that the argument passed to `useState` is only used the first time the component is rendered. On subsequent re-renders, the component keeps track of the existing state and does not need or use the initial value.

This is great—we now have a mechanism for both getting and setting the value of the changing state of our component, which we can then use through the rest of our component.

Let's jump to the JSX return value of the component. Two things about this value have changed:

- The `className` now includes a call to a `stateDisplayClass()` function.
- We've added another prop to the `span`, namely `onClick={changeState}`.

The `onClick` prop is how React does event handling: you create a prop whose name is `on` followed by the event; the value of that prop is a function that is called when the event happens. In our case, we're using `{changeState}`. (For a complete list of event names supported by React, check out the official docs on the React website.)[36]

When the button is clicked, the `onClick` event fires, which causes us to go to the `changeState` function inside our component. Within that function we do a check on the value of `status`—the same `status` variable that was defined by the call to `useState`. We then change the value of `status` based on the current value of `status` using the `setState` function, also the one defined by `useState`, to officially register the change with React.

Using `setState` triggers a redraw of the element, which takes us back to the return value and the call to `stateDisplayClass()`, which is used to change the background color of the item based on the current status. Clicking once changes the status to `held`, which then causes the display class to be `bg-green-500`—Tailwind-speak for "make the background green." Clicking again calls `setStatus("unsold")`, and the re-